

Python Tip: Truthy and Falsy Values

Have you ever seen an `if` statement evaluate a list or string? Those values aren't Booleans—but Python still treats them as true or false.

Let's find out how!



A List in an `if` Statement?

Here, `items` is a list. It is not a `True` or `False` Boolean value. So why doesn't this crash?

```
items = ["book", "pen"]  
  
if items:  
    print("Your cart has items")
```

Your cart has items

Python knows how to treat non-Boolean values inside conditions.

Truthy and Falsy

Python treats common non-Boolean objects as **truthy** or **falsey**.

```
full_cart = ["book", "pen"]  
empty_cart = []  
  
print("Full:", bool(full_cart))  
print("Empty:", bool(empty_cart))
```

```
Full: True  
Empty: False
```

A list is **truthy** when it contains items and **falsey** when it is empty.
This makes quick checks like `if items:` easy!

Common Truthy Values

Python usually treats these values as `True`:

- Nonzero numbers: `1`, `-1`, `3.14`
- Non-empty strings: `"hello"`, `"False"`
- Non-empty lists: `[1]`, `[False]`
- Non-empty dictionaries: `{"name": "Ada"}`

Python checks whether a collection is empty—not whether the values inside it are true or false.

Common Falsy Values

Python usually treats these values as `False`:

- The Boolean value `False`
- The missing value `None`
- Zero: `0`, `0.0`
- An empty string: `''`
- Empty collections: `[]`, `{}`, `set()`

Wrap-Up

Python treats values as **truthy** or **falsy** when they are used in a condition.

Zero, **None**, **False**, and empty values are usually falsy.

Most other values are **truthy**.

Use **bool(value)** to check a value's truthiness!

Follow me for more tips.

- Shep4